

Competitive Programming

Section 57 Editorial: Reductions And Modeling

Competitive Programming Course

2026-07-18

Table of contents

1	Editorial overview	3
1.1	Read before code: problem dossiers	3
1.2	Problem-specific proof map	3
1.3	Detailed solution guide	3
2	A. Matching Model	5
2.1	Restatement	5
2.2	Observations and algorithm	5
2.3	Correctness proof	5
2.4	Complexity	5
2.5	Full C++ solution	5
2.6	Full Python solution	5
3	B. Small 2-SAT	7
3.1	Restatement	7
3.2	Observations and algorithm	7
3.3	Correctness proof	7
3.4	Complexity	7
3.5	Full C++ solution	7
3.6	Full Python solution	7
4	C. Capstone Matching	9
4.1	Restatement	9
4.2	Observations and algorithm	9
4.3	Correctness proof	9
4.4	Complexity	9
4.5	Full C++ solution	9
4.6	Full Python solution	9
5	D. DAG Path Cover	11
5.1	Restatement and model	11
5.2	Trace and count identity	11
5.3	Algorithm	11
5.4	Correctness proof	12
5.5	Complexity, implementation notes, and tests	12
5.6	Full C++ solution	12
5.7	Full Python solution	13
6	What to carry forward	15

1 Editorial overview

1.1 Read before code: problem dossiers

A. Matching Model. Left vertices are requesters, right vertices are resources, and edges are legal assignments. A valid assignment is exactly a bipartite matching; augmenting paths improve it by one. Test competing left vertices and a rematching chain. Complexity $O(VE)$ for DFS matching.

B. Small 2-SAT. Clause $(a \text{ or } b)$ creates implications $\text{not } a \rightarrow b$ and $\text{not } b \rightarrow a$. A variable is impossible exactly when it and its negation share an SCC. Test a unit-like contradiction and a satisfiable cycle. Complexity $O(V+E)$.

C. Capstone Matching. Repeat the bidirectional modeling proof from A, then stress a graph where greedy first-edge assignment fails but an augmenting path finds the optimum.

1.2 Problem-specific proof map

A. Matching Model. Choosing disjoint compatible pairs is exactly a bipartite matching: each legal assignment is a matching and each matching is legal.

B. Small 2-SAT. Clause $(a \text{ or } b)$ is equivalent to implications $\text{not } a \rightarrow b$ and $\text{not } b \rightarrow a$. A variable is impossible precisely when it and its negation share an SCC.

C. Capstone Matching. Use the same bidirectional reduction as A, then test the matching implementation on an augmenting path that forces a reassignment.

D. DAG Path Cover. Copy every DAG vertex to a left and a right side; an edge $u \rightarrow v$ can join u immediately before v in a path. A matching chooses nonconflicting joins, so $n - \text{maximum_matching}$ is the minimum path count.

1.3 Detailed solution guide

A. Assignment as Matching. Create a left vertex for each worker and a right vertex for each task. Add an edge when that worker can perform that task. A matching chooses at most one incident edge per worker and per task, exactly matching the assignment constraints. Therefore a matching of size k corresponds to k valid assignments, and a maximum matching gives the maximum number of assignments.

B. 2-SAT as SCC. For every clause $(a \text{ or } b)$, add implications $\text{not } a \rightarrow b$ and $\text{not } b \rightarrow a$. If a variable and its negation are in the same SCC, each implies the other, so both truth values would be forced and the formula is impossible. Otherwise, SCC topological order gives an assignment: choose the later component between x and $\text{not } x$.

C. Modeling Capstone. The capstone asks for the same habit: identify the objects, identify incompatibilities or implications, then choose the graph model whose feasible solutions match

the original statement in both directions. The proof must explicitly map original solutions to graph solutions and graph solutions back to original solutions.

Common pitfalls. A reduction is not complete until it is bidirectional. Check that every graph solution satisfies all original constraints, not only that every original solution can be represented.

Each solution starts by building a model graph or implication system.

- A is direct bipartite matching: every selected edge pairs one left item with one right item, and matching enforces uniqueness.
- B builds implications for each 2-SAT clause. The formula is impossible iff a variable and its negation share an SCC.
- C reinforces the same matching reduction on a second instance.

The reduction proof must go both ways: every original solution maps to a model solution, and every model solution maps back to a valid original answer.

2 A. Matching Model

2.1 Restatement

Given a bipartite graph, print maximum matching size.

2.2 Observations and algorithm

Create a left vertex per requester and right vertex per resource; add an edge for each legal assignment. A matching is precisely a set of non-conflicting assignments. Repeated DFS finds augmenting paths; test two left vertices that initially want the same right vertex.

2.3 Correctness proof

An augmenting path starts at an unmatched left vertex and ends at an unmatched right vertex. Flipping its edges increases matching size by one; when none exists, Berge's theorem proves maximality. $\square$

2.4 Complexity

DFS matching is $O(VE)$ time and $O(V)$ memory. Reset right-side visited flags for every DFS root; otherwise valid reroutes are missed.

2.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int n,m,e;if(!0
```

2.6 Full Python solution

```
import sys, collections
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    n,m,e=data[0],data[1],data[2]; adj=[[] for _ in range(n)]; idx=3
    for _ in range(e): adj[data[idx]-1].append(data[idx+1]-1); idx+=2
```

```
match=[-1]*m
def dfs(u,vis):
    for v in adj[u]:
        if vis[v]: continue
        vis[v]=1
        if match[v]==-1 or dfs(match[v],vis): match[v]=u; return True
    return False
ans=0
for u in range(n): ans+=dfs(u,[0]*m)
print(ans)
if __name__=="__main__": main()
```

3 B. Small 2-SAT

3.1 Restatement

Given clauses $(a \text{ or } b)$ over signed variables, decide satisfiability.

3.2 Observations and algorithm

For clause $(a \text{ or } b)$, add implications $\text{not } a \rightarrow b$ and $\text{not } b \rightarrow a$. Build the implication graph and its reverse, then compute SCCs. A variable and its negation in one SCC force each other and make the formula impossible.

3.3 Correctness proof

Each implication encodes exactly the assignments forbidden by its clause. SCC co-membership gives paths both ways between a literal and its opposite, hence a contradiction; absence permits the standard reverse-SCC assignment. $\square$

3.4 Complexity

Kosaraju/Tarjan is $O(V+E)$ time and memory. Test (x) encoded as $(x \text{ or } x)$ and contradictory unit clauses.

3.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int n,m;if(!(ci
```

3.6 Full Python solution

```
import sys, itertools
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    n,m=data[0],data[1]; clauses=[]; idx=2
    for _ in range(m): clauses.append((data[idx],data[idx+1])); idx+=2
```

```

for mask in range(1<<n):
    ok=True
    for a,b in clauses:
        va=((mask>>(abs(a)-1))&1)==(a>0); vb=((mask>>(abs(b)-1))&1)==(b>0)
        if not (va or vb): ok=False; break
    if ok: print("YES"); return
print("NO")
if __name__=="__main__": main()

```

4 C. Capstone Matching

4.1 Restatement

Another matching instance to reinforce the reduction pattern.

4.2 Observations and algorithm

Use the same bipartite graph model as A, but first write the two directions of the reduction: every legal allocation is a matching and every matching is a legal allocation. Then run augmenting DFS.

4.3 Correctness proof

The matching proof from A applies unchanged. In particular, a greedy first-edge choice can fail, while an augmenting path repairs that choice. $\square$

4.4 Complexity

Complexity is $O(VE)$. Stress a chain that requires rematching two vertices.

4.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int n,m,e;if(!
```

4.6 Full Python solution

```
import sys, collections
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    n,m,e=data[0],data[1],data[2]; adj=[[] for _ in range(n)]; idx=3
    for _ in range(e): adj[data[idx]-1].append(data[idx+1]-1); idx+=2
    match=[-1]*m
    def dfs(u,vis):
```

```

    for v in adj[u]:
        if vis[v]: continue
        vis[v]=1
        if match[v]==-1 or dfs(match[v],vis): match[v]=u; return True
    return False
ans=0
for u in range(n): ans+=dfs(u,[0]*m)
print(ans)
if __name__=="__main__": main()

```

5 D. DAG Path Cover

5.1 Restatement and model

Given a directed acyclic graph, cover every vertex with the smallest possible number of vertex-disjoint directed paths. A path may have one vertex. There are at most 2,000 vertices and 10,000 edges, so trying every way to connect paths is not practical.

Create two copies of every vertex: `u_left` and `u_right`. For each DAG edge `u -> v`, add bipartite edge `u_left -> v_right`. Matching such an edge means we put `v` immediately after `u` in one path. Because matching uses each left and right vertex at most once, no original vertex receives two predecessors or two successors.

5.2 Trace and count identity

For edges `1->2`, `1->3`, `2->4`, `3->4`, a matching can select `1->2` and `2->4`. These joins make path `1->2->4`; vertex 3 remains a singleton. There are four vertices and two selected joins, hence two paths.

Starting from `n` singleton paths, every selected join merges two paths and reduces the count by one. Therefore a matching of size `M` produces `n-M` paths. We want as many joins as possible, so the answer is:

```
minimum path cover = n - maximum bipartite matching
```

The graph must be acyclic. Otherwise the selected joins could form a directed cycle, which is not a path cover.

5.3 Algorithm

```
build bipartite adjacency: u_left -> v_right for every original edge u -> v
run augmenting-path matching
print n - matching_size
```

The local limits permit the standard DFS augmenting algorithm. Reset the visited-right array for every new left-side root so an earlier failed search does not suppress a valid later reroute.

5.4 Correctness proof

Take any valid path cover with p paths. Each path of length L contributes $L-1$ consecutive original edges, and all these edges have distinct starts and distinct ends. They form a matching of size $n-p$ in the constructed bipartite graph. Hence $\text{maximum_matching} \geq n-p$, so $n - \text{maximum_matching} \leq p$.

Conversely, take any matching of size M . Each matched edge gives an original edge whose start has at most one selected successor and whose end has at most one selected predecessor. Thus selected edges form disjoint directed chains; acyclicity rules out a directed cycle. Add every unmatched vertex as needed as a singleton chain. These chains cover every vertex and contain exactly $n-M$ paths. Hence an optimal matching gives a path cover of size $n - \text{maximum_matching}$. Together, the two inequalities prove the formula and the algorithm. QED.

5.5 Complexity, implementation notes, and tests

With DFS augmenting paths, time is $O(n \cdot m)$ and memory is $O(n+m)$, suitable for the stated local bounds. Store the matching by right-side vertex. Test an edgeless graph (every vertex is a path), a chain (one path), a fork that cannot put both children after the same parent, and a diamond requiring a rematch.

5.6 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    if (!(cin >> n >> m)) return 0;
    vector<vector<int>>> graph(n);
    for (int i = 0; i < m; i++) {
        int from, to;
        cin >> from >> to;
        graph[from - 1].push_back(to - 1);
    }

    vector<int> match_right(n, -1);
    function<bool(int, vector<int>&)> augment = [&](int left, vector<int>& seen) {
        for (int right : graph[left]) {
            if (seen[right]) continue;
            seen[right] = true;
            if (match_right[right] == -1 || augment(match_right[right], seen)) {
                match_right[right] = left;
                return true;
            }
        }
    }
```

```

    }
    return false;
};

int matching = 0;
for (int left = 0; left < n; left++) {
    vector<int> seen(n, false);
    matching += augment(left, seen);
}
cout << n - matching << '\n';
return 0;
}

```

5.7 Full Python solution

```

import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n, m = data[0], data[1]
    graph = [[] for _ in range(n)]
    pointer = 2
    for _ in range(m):
        start, end = data[pointer] - 1, data[pointer + 1] - 1
        pointer += 2
        graph[start].append(end)

    match_right = [-1] * n

    def augment(left: int, seen: list[bool]) -> bool:
        for right in graph[left]:
            if seen[right]:
                continue
            seen[right] = True
            if match_right[right] == -1 or augment(match_right[right], seen):
                match_right[right] = left
                return True
        return False

    matching = 0
    for left in range(n):
        matching += augment(left, [False] * n)
    print(n - matching)

```

```
if __name__ == "__main__":  
    sys.setrecursionlimit(10_000)  
    main()
```

6 What to carry forward